





Bachelor Thesis

Opinion Dynamics in Networks of LLM Agents: A Comparison with Classical Rules

Theo Radicella
Artificial Intelligence

Academic Year 2026/2027

Bachelor in Artificial Intelligence
Pavia 2026

Bachelor in Artificial Intelligence







Theo Radicella

Opinion Dynamics in Networks of LLM Agents: A Comparison with Classical Rules

Supervisors: Flaminio Squazzoni, University of Milan
Stefano Zapperi, University of Milan
Yick Chung (William), University of Milan
Francesco Renzini, University of Milan

© Theo Radicella, 2026

Contents

1 Introduction **3**
- 1.1 Motivation . . . 3
- 1.2 Problem formulation and research questions . . . 3
- 1.3 Contribution . . . 4
- 1.4 Main results . . . 5
- 1.5 Outline . . . 5

2 Background and Related Work **6**
- 2.1 Agent-based models and networks . . . 6
  - 2.1.1 Describing a network . . . 6
  - 2.1.2 Random networks: Erdős–Rényi . . . 7
  - 2.1.3 Small-world networks: Watts–Strogatz . . . 7
  - 2.1.4 Scale-free networks: Barabási–Albert . . . 8
  - 2.1.5 Comparing the three topologies . . . 9
- 2.2 Numeric opinion dynamics . . . 10
  - 2.2.1 DeGroot: plain averaging . . . 11
  - 2.2.2 Bounded confidence . . . 11
  - 2.2.3 Friedkin–Johnsen: stubbornness . . . 12
- 2.3 Polarization and echo-chamber metrics . . . 13
- 2.4 LLM agents in social simulation . . . 14
- 2.5 Relation to this thesis . . . 15

3 Method: The disensa Framework **17**
- 3.1 Simulation loop . . . 17
- 3.2 Networks and initial opinions . . . 18
  - 3.2.1 Networks . . . 18
  - 3.2.2 The networks of the experiments . . . 18
  - 3.2.3 Initial opinions . . . 19
- 3.3 Numeric agents . . . 20
- 3.4 LLM agents . . . 20
  - 3.4.1 What an agent is . . . 21
  - 3.4.2 Step 0: the first message . . . 21
  - 3.4.3 Interactions . . . 22
  - 3.4.4 Updating the opinion . . . 23
  - 3.4.5 Model assignment . . . 23
- 3.5 Seeding and reproducibility . . . 24
- 3.6 Malformed output and the run report . . . 25
  - 3.6.1 Reading a reply . . . 25
  - 3.6.2 Why failures need a limit . . . 26
  - 3.6.3 The run report . . . 26
- 3.7 Measuring opinions: self-rating, scorer, human validation . . . 27

3.8 Comparing LLM runs with numeric rules . . . 27
3.9 Metrics . . . 27
3.10 Experimental design . . . 27
3.11 Computational setup . . . 27

4 Experiments and Results **28**
4.1 Design overview . . . 28
4.2 RQ A: how far do the classic rules describe LLM runs? . . . 28
4.3 RQ B: how do individual agents update their opinions? . . . 28
4.4 Numeric baseline sweeps . . . 28

5 Discussion and Limitations **29**
5.1 Interpretation of the results . . . 29
5.2 Self-rating versus scorer . . . 29
5.3 Limitations . . . 29
5.4 Reproducibility . . . 30

6 Conclusion **31**
6.1 Review of Project Goals . . . 31
6.2 Context . . . 31
6.3 Future Works . . . 31

A Prompts **32**

B Configurations and Seeds **33**

Chapter 1
Introduction

1.1 Motivation

Many collective phenomena are studied with **agent-based models**: a population of agents, each following a simple local rule, whose behaviour as a whole emerges from their interactions rather than being written into any single rule [25]. Opinion dynamics is one of the oldest applications of this idea. Given a simple rule for how one person updates an opinion after hearing from others, what pattern appears once a whole network of people follow it together? Do they converge on a shared view, split into opposing camps, or stay scattered? Three classic rules of this kind (DeGroot, bounded confidence, Friedkin–Johnsen) are each simple enough to write as one equation, and each is already well understood analytically (Chapter 2).

Large language models complicate this picture in an interesting way. An LLM agent does not apply an equation: it is prompted with what it has heard and asked to decide on a new opinion itself, in language, with whatever reasoning the model produces along the way [23, 8]. The local rule is no longer visible or fixed; it is whatever the model does, and it can differ from one language model to the next. This is what makes the population-level question interesting again for a case where the local rule is not already known: to what extent can the runs of a population of LLM agents be described by the classic rules, even though none of the agents was told to follow one, and where do they behave differently from all of them? And, since each LLM agent is more than a single number, unlike a numeric agent, can the reasoning it produces along the way explain *why* it moved, not only that it did? These two questions, about the population as a whole and about a single agent, are what this thesis sets out to answer for a working implementation, disensa, described in Chapter 3.

1.2 Problem formulation and research questions

disensa runs the same simulation loop over a network of agents, with two interchangeable kinds of agents: **numeric agents**, which apply one of the three classic rules of Chapter 2 directly to their neighbours' opinions, and **LLM agents**, which read the messages their neighbours shared and decide on a new opinion, a new message and an updated memory themselves (Chapter 3). Because both kinds of agents can run on the exact same network and the exact same starting opinions, an LLM run and a numeric run can be compared directly, state for state, rather than only by their general shape.

This thesis asks two questions on top of that comparison, one about the whole population and one about a single agent:

- **RQ A** (the population): to what extent can LLM-agent runs be described by the classic opinion-dynamics rules (DeGroot, bounded confidence, Friedkin–Johnsen), and does this vary with the topic type (political, moral, factual, trivial) or with the network topology (random, small-world, scale-free)?

- **RQ B** (one agent): how do individual LLM agents update their opinions in response to their neighbours, and what in their logged interactions and reasoning explains a change?

For RQ A, "described by" has to be defined before the results are seen. Otherwise the comparison could be chosen afterwards to favour one answer. First, the LLM agents run on a network with a set of starting opinions, for a fixed number of steps. Then the same run is repeated with the LLM agents replaced by numeric agents following one of the three classic rules: the same network, the same starting opinions and the same number of steps. Each rule is run with many values of its parameters (for example, the confidence bound of bounded confidence), and the values that bring it closest to the LLM run are kept. The two runs are then compared on several aspects: the final opinion distribution (Wasserstein distance), the number of opinion clusters, the drift of the mean opinion and the polarization over the steps (Section 3.8). A distance on its own does not say whether a rule describes a run well, so it is read against two references. At one end is a baseline in which nobody moves: the final opinions are simply the starting ones. A rule that is not closer to the run than this describes nothing, and has to beat it to count at all. At the other end is a replicate: the same LLM agents run again on the same world (done for one of the models, Llama 3.1 8B; Section 3.10). The two LLM runs differ only through the randomness of the model, so their distance is about as close as any rule could be expected to get. The closer a rule comes to the replicate, the better it describes the run. Two things follow. First, no rule has to fit: if none beats the baseline, the answer is that the classic rules do not describe that run. Second, the rules are not competitors from which one winner must be chosen. They overlap: bounded confidence with a bound wide enough to include everyone and full step size ($\mu = 1$) *is* DeGroot, and Friedkin–Johnsen with no stubbornness is plain averaging over the neighbours. Two rules can therefore describe a run equally well, and each rule's distance is reported, not only the closest.

RQ B looks at the same agents without assuming any rule. At every step, each agent's move is set against the gap between its own opinion and the opinions it heard. The resulting curve, of how far agents move as that gap grows, shows how LLM agents actually respond. Each classic rule predicts a shape for this curve (DeGroot, for instance, a straight line: the bigger the gap, the bigger the move), and these predictions are drawn next to it only for comparison. An agent may also move without hearing anyone, for example by drifting towards the model's own view of the topic, so runs in which agents hear nobody measure that drift, and what remains in the normal runs is what the interactions add. Finally, the logged reasoning of single agents shows what they responded to. A numeric rule explains its own moves for free (its equation *is* the explanation); an LLM agent does not, unless its interactions are logged and inspected directly. Asking both questions of the same runs is only possible because every LLM run disensa produces is fully logged at the level of each agent and each step (Chapter 3).

1.3 Contribution

This thesis contributes:

- disensa, a simulation framework that runs LLM-agent and numeric-rule opinion dynamics through the same loop, on the same networks, with the same seeding, so that an LLM run and the numeric-rule runs it is compared with start from the exact same world and can be compared state for state rather than only in their general shape (Chapter 3);
- full per-agent, per-step logging of every LLM run (interactions, reasoning, message and memory), making it possible to trace a single agent's opinion change and inspect its reasoning directly, which is what RQ B asks for;

- a methodology for measuring how far the three classic rules describe an LLM run on its own world: each rule is fitted to the run's network and starting opinions and compared with it on four aspects (the final opinion distribution, the number of clusters, the drift of the mean opinion and the polarization over the steps), against a baseline in which nobody moves and a replicate of the LLM run itself (Section 3.8);
- a methodology for measuring what an LLM agent's opinion actually is, combining its own self-rating with a separately validated scorer and blind human ratings (Section 3.7).

disensa also supports placing several language models side by side as agents of the same run. This thesis does not use that capability: every run is driven by a single model, and four models are compared across separate runs (Section 3.10). Mixed-model runs are left for future work (Section 2.5 explains why). RQ A and RQ B, run on top of the methodology above, are what the rest of this thesis reports on.

1.4 Main results

[**TODO: fill in once Chapter 4 is complete.**] One short paragraph each for RQ A and RQ B, stating the headline finding, written only once there are actual results to report.

1.5 Outline

The rest of this thesis is organised as follows:

Chapter 2 gives the background this thesis builds on: agent-based models and the three network topologies disensa uses, the three classic numeric opinion-dynamics rules, the metrics used to read polarization and echo chambers off a run, the LLM-agent social simulations closest to this one, and how disensa relates to them.

Chapter 3 describes disensa itself: the simulation loop; how networks and initial opinions are built; the numeric agents; the LLM agents, their prompts and how models are assigned to them; how randomness is seeded so runs are reproducible; how malformed model output is handled and reported; how an agent's opinion is measured, from its own self-rating to a separate scorer validated against human raters; how an LLM run is compared against the numeric rules; the metrics; the experimental design; and the computational setup used to run it.

Chapter 4 reports the experiments run to answer RQ A and RQ B, and their results.

Chapter 5 discusses what the results mean, how far they can be trusted, and the limitations of this thesis, including the risk that what a run of LLM agents shows reveals more about the language model than about the subject being simulated.

Chapter 6 reviews whether the goals of this thesis were met, places the work back in its wider context, and suggests directions for future work.

Appendix A quotes every prompt disensa sends to a language model, verbatim. Appendix B lists the configuration files and seeds behind every run reported in Chapter 4.

Chapter 2
Background and Related Work

This chapter gives the background needed for the rest of the thesis. Section 2.1 introduces agent-based models and the three network topologies disensa builds on. Section 2.2 covers the three numeric opinion-dynamics rules disensa implements as a reference for LLM agents. Section 2.3 defines the metrics used to read polarization and echo chambers off a run. Section 2.4 reviews the LLM-agent social simulations closest to this thesis. Section 2.5 places disensa among them.

2.1 Agent-based models and networks

An **agent-based model** (ABM) simulates a system as a population of individual agents, each following its own, usually simple, rule, and lets the behaviour of the whole population emerge from their interactions rather than specifying it directly [25]. A standard illustration is a colony of ants foraging for food: no single ant knows where the food is or plans a route to it. Each ant only follows pheromone trails and leaves its own trail behind. Yet the colony as a whole finds food efficiently, because the trails that lead to food get reinforced by every ant that follows them, and the ones that do not fade away. The efficient foraging pattern is not written into any ant's rule; it only appears once many ants run their simple rule at once, on top of each other's trails [25]. This is the kind of **emergent behaviour** ABMs are built to produce and study: outcomes at the level of the population that are not a property of any single agent's rule.

Opinion dynamics is a long-standing application of this idea. The agents are people (or, in this thesis, LLM agents standing in for them), the local rule is how an agent updates its view after hearing from others, and the population-level outcome under study is whether the group reaches consensus, splits into camps, or stays scattered. Section 2.2 covers three classic rules of this kind. All three, and disensa's LLM agents in Chapter 3, share one more structural ingredient: a **network**. The network fixes who can influence whom: an agent does not hear from the whole population, only from the agents it is linked to, its **neighbours**, and it updates its opinion using theirs alone. An opinion can therefore reach a distant agent only step by step, passing through the agents in between. This is why the shape of the network, and not only the update rule, affects how opinions spread and where they settle.

2.1.1 Describing a network

A network has N **nodes** (here, agents) and L **links**. This thesis only uses **undirected**, **unweighted** networks: a link between two nodes means those two agents interact, the link counts the same both ways, and it carries no strength. A network is stored as its **adjacency matrix** A, with $A_{ij} = 1$ if i and j are linked and 0 otherwise; for an undirected network A

is symmetric. The **neighbours** of node i are the nodes it is linked to, $N(i) = \{\, j : A_{ij} = 1 \,\}$. A node's **degree** k_i is its number of links, that is, its number of neighbours, $k_i = |N(i)|$:

$$k_i = \sum_j A_{ij}, \tag{2.1}$$

and the **average degree** is $\langle k \rangle = 2L/N$ (the factor 2 appears because every link raises the degree of both of its ends). The **degree distribution** p_k is the share of nodes with exactly k links: the main property that tells the three network topologies below apart. In particular, it shows whether the network has **hubs**: a few nodes with far more links than the average. The **clustering coefficient** of node i,

$$C_i = \frac{2L_i}{k_i(k_i - 1)}, \tag{2.2}$$

is the share of i's neighbours that are themselves linked to each other (L_i is the number of such links); averaged over all nodes it gives $\langle C \rangle$. The **distance** d_{ij} between two nodes is the number of links on the shortest path between them; averaged over all pairs it gives the **average path length** $\langle d \rangle$. A network is **connected** when a path exists between every pair of nodes; if it is not, groups of agents with no path between them can never influence each other. A network is **sparse** when L is far below the maximum possible number of links, $N(N-1)/2$.

2.1.2 Random networks: Erdős–Rényi

In the Erdős–Rényi model [13], every pair of the N nodes is linked independently with the same probability p, so

$$\langle k \rangle = p\,(N - 1). \tag{2.3}$$

Each node has $N - 1$ possible links, each present independently with probability p, so its degree follows a **binomial distribution**. When the network is sparse ($\langle k \rangle$ much smaller than N), the binomial is very close to a **Poisson distribution** with mean $\langle k \rangle$,

$$p_k = e^{-\langle k \rangle} \frac{\langle k \rangle^k}{k!}, \tag{2.4}$$

which is the form usually quoted, because it depends on $\langle k \rangle$ alone and not on N and p separately [1]. With either form, the probability of a degree far above $\langle k \rangle$ drops off very quickly, so an Erdős–Rényi network has **no hubs**: almost every node has close to the average number of neighbours. Its clustering is low and shrinks as the network grows, $\langle C \rangle \approx \langle k \rangle / N$, since any two neighbours of a node are linked with the same probability p as any other pair. Average distances, instead, are short. Starting from any node, its $\langle k \rangle$ neighbours are one link away, their neighbours (about $\langle k \rangle^2$ nodes) are two links away, and so on: every extra link multiplies the number of nodes reached by about $\langle k \rangle$. Because links are random, these are mostly nodes not reached before, so a path of a few links is enough to reach the whole network: about three for $N = 1000$ and $\langle k \rangle = 10$. For the same reason, distances grow only very slowly with the size of the network, as $\ln N$ [1]: doubling N adds less than one link to the average distance.

Because it has neither of the two structural features the other two topologies add (clustering, hubs), this network is used as the **null model**: a baseline against which the effect of adding structure can be measured.

2.1.3 Small-world networks: Watts–Strogatz

Real social networks tend to combine two properties. Like an Erdős–Rényi network, they have the **small-world property**: the typical distance between two nodes grows only logarithmically with the size of the network, so everyone is a few links from everyone. Unlike

it, they are also **clustered**: a node's neighbours tend to know each other. Watts and Strogatz [24] build a network that sits between two extremes. At one end is a **ring lattice**: the nodes are placed on a ring and each is linked to its k nearest neighbours ($k/2$ on each side). A node's neighbours are then mostly neighbours of each other, so clustering is high, but reaching the opposite side of the ring takes many links, so paths are long. At the other end is a random network: short paths, but almost no clustering. The construction starts from the ring lattice and rewires each link, independently, with probability β, moving one of its ends to a node chosen uniformly at random. At $\beta = 0$ nothing is rewired and the network stays the ring lattice: highly clustered, but distances grow linearly with N. At $\beta = 1$ it is close to an Erdős–Rényi network: low clustering, short paths. For small β (of the order of 0.01 to 0.1), a handful of rewired links act as shortcuts across the ring: the average distance $\langle d \rangle$ collapses quickly, while most of the original triangles survive and clustering stays close to its value at $\beta = 0$, roughly [3]

$$C(\beta) \approx C(0)\,(1-\beta)^3. \tag{2.5}$$

This is the **small-world regime**: short paths and high clustering together. Rewiring moves only one end of a link, so degrees stay close to k throughout and this network, like the Erdős–Rényi one, has **no hubs**. Figure 2.1 shows the small-world regime for networks of the size used in this thesis.

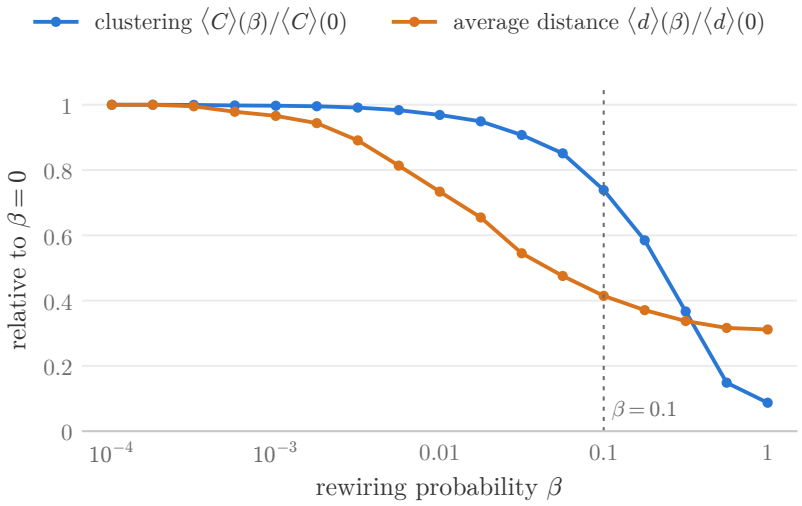


Figure 2.1: Average clustering $\langle C \rangle$ and average distance $\langle d \rangle$ of a Watts–Strogatz network as the rewiring probability β grows, each divided by its value on the ring lattice ($\beta = 0$), on a logarithmic β axis. $N = 100$, $k = 6$; each point is the mean over 40 networks built by disensa. Distances fall well before clustering does: between these two drops lies the small-world regime. The dotted line marks $\beta = 0.1$, the value used in this thesis's configurations. After Watts and Strogatz [24], who used $N = 1000$, where the effect is sharper.

2.1.4 Scale-free networks: Barabási–Albert

Many real networks also have hubs: a few nodes with far more links than the rest, while most nodes have only a few. Barabási and Albert [2] reproduce this with a degree distribution that follows a power law, $p_k \sim k^{-\gamma}$, rather than the Poisson distribution of the random network. A power law decays slowly enough that very high degrees remain possible, if rare: there is no typical value of k, which is what **scale-free** refers to. The construction has two ingredients, both necessary: **growth** (nodes are added one at a time, never removed) and **preferential attachment**: each new node arrives with m links, and each of them goes to an

existing node with probability proportional to that node's current degree. The probability that a link of the new node attaches to the existing node i is

$$\Pi_i = \frac{k_i}{\sum_j k_j}, \tag{2.6}$$

where the sum runs over all existing nodes, so the probabilities add up to 1. For example, if the existing nodes have degrees 1, 2, 3 and 4, the node with degree 4 receives the link with probability 4/10, and the node with degree 1 with probability 1/10. Better-connected nodes are more likely to gain the next link, so they tend to become even better connected; because nodes are added over time, the oldest nodes have had the most chances to do so and become the hubs (a *first-mover advantage*). Growth without preferential attachment gives an exponential degree distribution and no hubs; preferential attachment without growth gives no stable power law. Together, they produce

$$p_k \approx 2m(m+1)\,k^{-3}, \tag{2.7}$$

a power law with exponent $\gamma = 3$ regardless of m, and average degree $\langle k \rangle \approx 2m$. Clustering is low, vanishing as $\langle C \rangle \sim (\ln N)^2/N$, and distances are even shorter than in the random network, $\langle d \rangle \sim \ln N / \ln \ln N$ at $\gamma = 3$ [1]. Figure 2.2 compares this power law with the Poisson distribution of a random network with the same average degree.

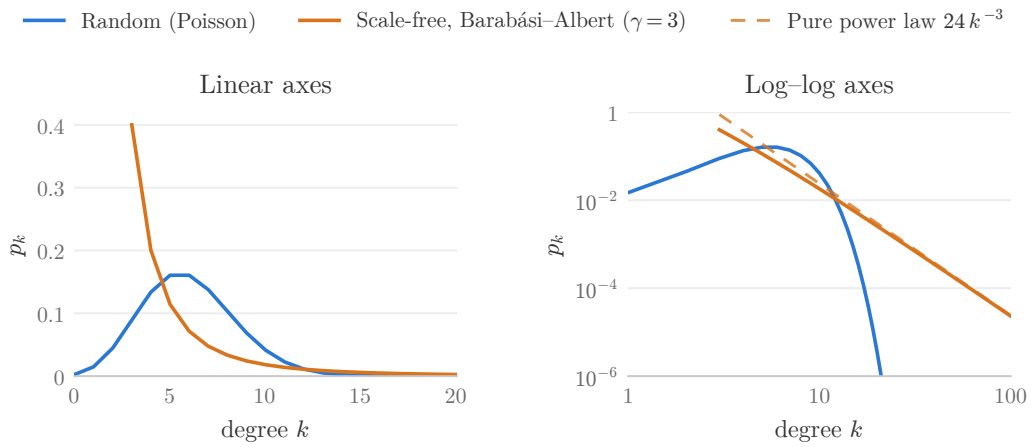


Figure 2.2: Degree distribution p_k of a random (Erdős–Rényi) and a scale-free (Barabási–Albert, $m = 3$) network with the same average degree, $\langle k \rangle = 6$. On linear axes (left) both tails seem to vanish past $k \approx 15$. On log–log axes (right) the difference shows: the Poisson tail collapses, while the power law falls along a straight line of slope -3, so very high degrees stay orders of magnitude more likely. The dashed line is the pure power law $2m(m+1)\,k^{-3}$ of Equation (2.7): the exact curve starts slightly below it and settles onto it as k grows. The scale-free network also has many more nodes with few links: it starts at $k = m = 3$, where its peak is.

2.1.5 Comparing the three topologies

Table 2.1 summarises the three topologies, and Figure 2.3 shows one network of each. Each isolates one structural property of real social networks while keeping paths short, so the effect of that property alone can be studied by comparing runs across topologies with the same number of agents and the same average degree: the random network is the baseline where structure plays no role, the small-world network adds clustering, and the scale-free network adds hubs. All three results are asymptotic, for large N; at the network sizes used in this thesis (tens of agents) they hold only approximately, and a scale-free network of that size shows visible hubs without a clean power-law tail.

Table 2.1: Structural properties of the three network topologies, all else (nodes, average degree) held equal.

	Clustering	Hubs	Short paths
Random (Erdős–Rényi)	low	no	yes
Small-world (Watts–Strogatz)	high	no	yes
Scale-free (Barabási–Albert)	low	yes	yes (even shorter)

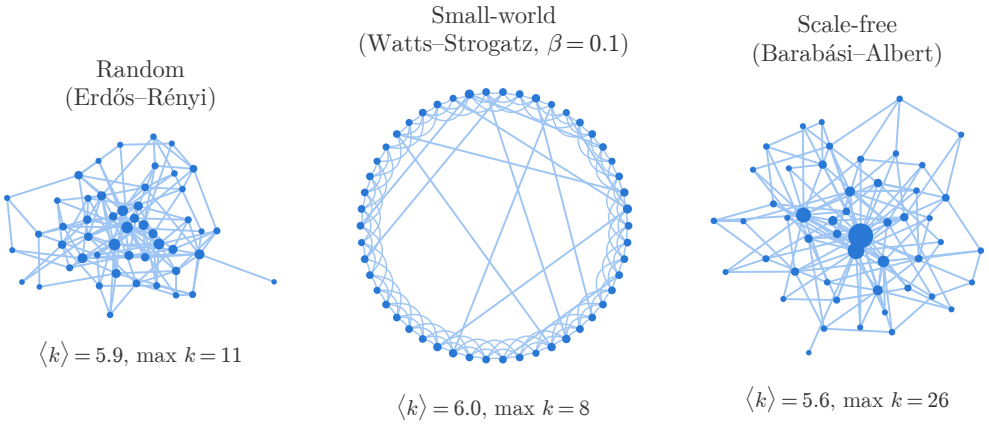


Figure 2.3: One network of each topology, built by disensa with $N = 50$ and average degree about 6 ($p = 0.12$; $k = 6$, $\beta = 0.1$; $m = 3$). Node size grows with the square of the degree, the same in all three panels. The random network has no structure beyond chance. The small-world network is drawn on its ring: the arcs along the ring are the lattice's local links, and the straight lines across it are the rewired shortcuts. The scale-free network has a few hubs, the largest with 26 links, more than twice the largest degree in the other two.

All three networks in this thesis are built with NetworkX [18], a Python library for generating and analysing graphs, and are detailed in Section 3.2.

2.2 Numeric opinion dynamics

On top of a network, an **opinion-dynamics** model decides how each agent's opinion changes from hearing its neighbours. Every agent i holds an opinion $x_i(t) \in [-2, 2]$ at step t (-2 strongly against a statement, $+2$ strongly in favour), and together they form the vector $x(t)$. Updates are **synchronous**: every agent computes $x_i(t)$ from the state $x(t-1)$, and all agents update at once, so the order in which agents are processed never matters. The three rules below all belong to the family of **assimilative influence** models, where an agent's opinion only ever moves towards the opinions it listens to, never away from them [14]; models that also allow repulsion are a separate family, not used in this thesis. Flache et al. [14] survey this broader space of models and the open questions around it; the three rules below are the classic, most frequently used members of the assimilative family. This thesis does not assume that LLM agents follow any of them: the three rules serve as a well-understood reference against which LLM agents are measured (Section 1.2).

2.2.1 DeGroot: plain averaging

The earliest and simplest rule, due to DeGroot [12], has every agent replace its opinion with the average of its own opinion and its neighbours' at the previous step,

$$x_i(t) = \frac{1}{k_i + 1} \sum_{j \in N(i) \cup \{i\}} x_j(t-1), \tag{2.8}$$

where $N(i)$ is the set of i's neighbours (Section 2.1.1). Every opinion in the average, including the agent's own, counts the same. Written for the whole population at once, this is a single matrix multiplication, $x(t) = Wx(t-1)$, with W a **row-stochastic** matrix (every entry is ≥ 0 and every row sums to 1): a general property of DeGroot-style models is that any such W gives a weighted-average update, with the equal weighting above as one special case. On a connected network, repeated averaging always converges to a single **consensus** value. This value is not the plain average of the initial opinions: even though every agent weighs the opinions it hears equally, a well-connected agent is heard by more agents, so its opinion enters more averages ($k_i + 1$ of them, counting its own). The consensus is therefore a weighted average of the initial opinions with weights proportional to $k_i + 1$, and better-connected agents pull it further towards their own starting opinion [12]. Golub and Jackson [17] study when this kind of repeated averaging converges to the population's true belief (in their language, when "naive" learning is nonetheless "wise"), and show the answer depends on whether influence in the network concentrates in a few hubs or stays spread out. Because every update is an average, DeGroot opinions never leave the range of the initial ones, and the model can never produce anything other than consensus: it is the reference point against which the two rules below are compared, since whatever else a model produces must come from a mechanism DeGroot lacks.

2.2.2 Bounded confidence

Bounded-confidence models add a single mechanism to averaging: an agent only listens to neighbours whose opinion is already close enough to its own, within a **confidence bound** ε. Two standard versions exist. Deffuant et al. [11] update one randomly chosen pair of agents at a time, each moving a fraction μ of the way towards the other if they are within ε. Hegselmann and Krause [19] instead update every agent at once, each jumping straight ($\mu = 1$) to the plain average of every other agent within ε (in their original formulation, with no network: every agent can potentially hear every other). This thesis uses the variant analysed by Chazelle and Wang [6], who call it **inertial Hegselmann–Krause**: like Hegselmann and Krause, every agent updates at once by averaging over everyone within ε, but, like Deffuant et al., it moves only a fraction μ of the way there rather than jumping to it. Restricted to a network, agent i's **confidence set** at step $t-1$ is

$$S_i(t-1) = \{\, j \in N(i) \cup \{i\} : |x_i(t-1) - x_j(t-1)| \leq \varepsilon \,\}, \tag{2.9}$$

and the update is

$$x_i(t) = x_i(t-1) + \mu \left(\frac{1}{|S_i(t-1)|} \sum_{j \in S_i(t-1)} x_j(t-1) \; - \; x_i(t-1) \right). \tag{2.10}$$

Here $0 < \mu \leq 1$, and since opinions lie in $[-2, 2]$, the useful range of the bound is $0 < \varepsilon < 4$: at $\varepsilon \geq 4$ nobody is ever excluded. Much of the literature uses opinions on $[0, 1]$, so its values of ε correspond to values 4 times larger here.

Whether the population reaches consensus or splits depends on ε relative to how spread out opinions are: a large enough ε eventually links everyone through chains of mutually confident agents and the population converges, as with DeGroot; a small ε leaves separate **opinion clusters** that no longer listen to each other, and the smaller ε is, the more and smaller the clusters [14]. Hegselmann and Krause [19] describe three regimes as ε grows:

plurality (many surviving opinions), polarization (two camps), and consensus. On a network, clusters can also be shaped by the network itself, independently of ε: an agent can only ever be pulled by its own neighbours, so groups the network keeps apart can stay apart even at a large ε. As with DeGroot, every update is an average, so opinions never become more extreme than the initial ones: bounded confidence can split a population, but it cannot manufacture new extremists.

2.2.3 Friedkin–Johnsen: stubbornness

The third rule, due to Friedkin and Johnsen [16], builds on an older idea from French [15]: an agent's social influence depends on how connected it is, but an agent may also resist influence altogether by staying anchored to its own view. Each agent i keeps its initial opinion $s_i = x_i(0)$ fixed as its **innate opinion**, and its *expressed* opinion $x_i(t)$ is a mix of s_i and the current average of its neighbours,

$$x_i(t) = \frac{\lambda\, s_i + \sum_{j \in N(i)} x_j(t-1)}{\lambda + k_i}, \tag{2.11}$$

following the formulation used by Chitra and Musco [7]. The parameter λ (**stubbornness**) weighs the innate opinion against one neighbour; the more neighbours an agent has, the smaller a share of the update its own innate opinion gets, so well-connected agents are pulled harder by the group around them. Because every agent keeps being pulled back towards s_i, the population does not converge to a shared consensus: instead, each agent approaches its own **equilibrium** x_i^*, the point where the pull of s_i and the pull of its neighbours balance. Neighbours end up closer to each other than they started, but disagreement across the whole population never fully disappears: a large λ keeps agents near where they started, a small one brings the population close to the DeGroot consensus. The equilibrium can be computed in closed form from the **graph Laplacian** $\mathcal{L} = D - A$ (where D is the diagonal matrix of degrees), without simulating any steps, as

$$x^* = \lambda\,(\lambda I + \mathcal{L})^{-1}\, s, \tag{2.12}$$

since at equilibrium each agent's pull from its innate opinion and its pull from its neighbours' current average must cancel exactly. In the original model of Friedkin and Johnsen [16], every agent has its own stubbornness; here, as in Chitra and Musco [7], a single λ is shared by all agents (Chitra and Musco fix $\lambda = 1$; this thesis keeps it as a parameter).

Figure 2.4 shows the three rules on the same network and from the same initial opinions, so that only the rule differs.

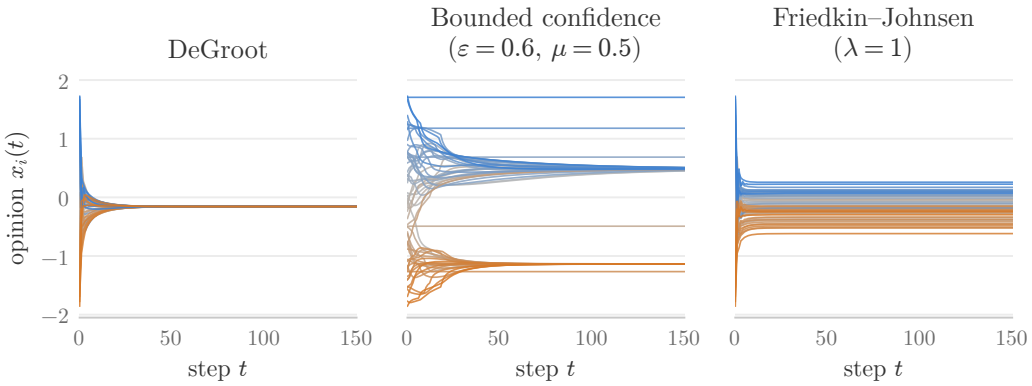


Figure 2.4: Every agent's opinion over the steps under the three rules, run by disensa on the same small-world network ($N = 60$, $k = 6$, $\beta = 0.15$) from the same uniformly drawn initial opinions. Colour shows each agent's initial opinion. DeGroot reaches a single consensus. Bounded confidence splits the population into two camps, plus a few agents whose neighbours all hold opinions more than ε away from their own, so they never move. Friedkin–Johnsen settles quickly, but each agent stays at its own equilibrium, so disagreement remains.

2.3 Polarization and echo-chamber metrics

Reading a run's outcome off the raw opinion vector $x(t)$ is only practical for very small networks. This thesis follows Wang et al. [23] (§4.2) in summarising every step with three numbers taken from the polarization literature: one that looks only at the spread of opinions, and two that also look at who is linked to whom. Read together, they are used to detect echo chambers, described at the end of this section.

Polarization P_z is the variance of the opinion vector across the whole population, the definition used by Musco et al. [21] and Chitra and Musco [7],

$$P_z = \frac{1}{N}\sum_{i=1}^{N}(x_i - \bar{x})^2, \qquad \bar{x} = \frac{1}{N}\sum_{i=1}^{N} x_i. \tag{2.13}$$

It is 0 at consensus and largest when the population splits evenly between the two extremes of the opinion scale. Because it is computed from the opinions alone, P_z is unchanged by shuffling which agent holds which opinion across the network: it says nothing about whether disagreement follows the network's structure.

Global disagreement DG fixes that by looking only at linked pairs. Musco et al. [21] define disagreement as the squared opinion difference summed over all links; Wang et al. [23] use a per-agent form of it. Each agent's **local disagreement** is the mean squared difference to its neighbours' opinions, and the global value averages this over the whole population,

$$DG_i = \frac{1}{k_i}\sum_{j \in N(i)} (x_i - x_j)^2, \qquad DG = \frac{1}{2N}\sum_{i=1}^{N} DG_i. \tag{2.14}$$

DG is 0 whenever every agent agrees with all of its neighbours, even if the population as a whole remains split into disagreeing groups. The factor $\frac{1}{2}$ makes DG comparable to P_z. For two agents picked at random, the expected squared difference of their opinions is twice the variance, $2P_z$. So if opinions were placed on the network at random, each DG_i would be about $2P_z$ and DG about P_z. A DG well below P_z therefore means that linked agents agree more than chance would predict: disagreement is *not* random with respect to the network.

Neighbour correlation NC, introduced by Cinus et al. [10], compares each agent's own opinion x_i with the average opinion of its neighbours, $y_i = \frac{1}{k_i}\sum_{j\in N(i)} x_j$. Every agent gives one pair (x_i, y_i), and NC is the Pearson correlation of these N pairs,

$$NC = \frac{\frac{1}{N}\sum_{i=1}^{N}(x_i - \bar{x})(y_i - \bar{y})}{\sigma_x\,\sigma_y}, \tag{2.15}$$

where $\bar{x}$ is the mean opinion of Equation (2.13), $\bar{y}$ is the mean of the y_i, and σ_x and σ_y are the standard deviations of the x_i and of the y_i (so $\sigma_x^2 = P_z$). The numerator is positive when agents and their neighbours tend to lie on the same side of the mean; the denominator only rescales the result to $[-1, 1]$. Like P_z and DG, NC is one number for the whole population, not a value per agent. A value near 0 means an agent's opinion says nothing about its neighbours'; a value close to $+1$ means agents are systematically surrounded by others who think like them.

In the literature, these metrics are used to detect echo chambers. An **echo chamber** is an environment in which individuals mostly meet opinions that agree with their own, so that their beliefs are reinforced rather than challenged. Cinelli et al. [9] make this measurable as the coexistence of two ingredients: (i) opinions are *polarized*, and (ii) interactions are *homophilic*, meaning individuals are mostly linked to others who think like them. Both are needed: a population split in two, with every agent linked across the divide, is polarized but has no echo chamber, and a tightly linked group in a population that agrees has nothing to be shielded from. An echo chamber is therefore not the same as the clustering of Section 2.1.1, which describes the links alone and says nothing about opinions.

Each metric captures only one ingredient, so no single one of them identifies an echo chamber; Wang et al. [23] make the same point and read the three side by side. P_z measures ingredient (i), and DG and NC measure ingredient (ii). This thesis reads them as follows. High P_z, DG well below P_z and high NC mean that opinions are split and sorted along the network into agreeing groups: an echo chamber. High P_z with DG close to P_z and NC near 0 means opinions are split but scattered over the network, with no structure. P_z and DG both near 0 mean consensus (NC is then undefined, since there is no variation left to correlate). Wang et al. find the first pattern in their simulations on small-world and scale-free networks, but not on random graphs.

2.4 LLM agents in social simulation

A recent line of work replaces the fixed update equations of Section 2.2 with a large language model: instead of computing a new opinion from a formula, each agent is prompted with what it has heard and asked to decide on a new opinion itself. This section reviews four studies close to this thesis, in the order they were published: three that simulate opinion change among LLM agents, and one that uses LLMs to measure real public opinion.

Chuang et al. [8] simulate small populations of LLM agents (10 in most experiments, 20 in a robustness check), driven by GPT-3.5 and checked with GPT-4 and Vicuna-33B. Each agent role-plays a persona: a name, political leaning, age, gender, ethnicity, education and occupation. Despite the paper's title, the agents are not placed on a network structure: at each time point a pair of agents is drawn uniformly at random, one writes a short message about the topic and the other reacts to it in words. A separate classifier (Flan-T5-XXL) reads that verbal reaction and converts it into an opinion on a five-point scale from -2 to $+2$; the authors chose this because they found the classifier's ratings more reliable than the agents' own self-reported numbers. The classifier's rating is the opinion the simulation keeps, not only a later measurement. Their main methodological contribution for this thesis is the **validation** of this scorer: three of the authors independently rated 100 agent responses without seeing the classifier's output, agreeing with each other at Krippendorff's $\alpha = 0.81$, and the classifier matched their majority judgement 84% of the time. Across 15 topics, their agents' opinions drifted towards the scientifically accurate side of factual claims, whatever the

starting distribution of opinions; only when the prompt explicitly introduced a confirmation bias did opinions fragment instead of converging. They established this with two design choices. Every topic was stated in a true and a false framing (for example, that global warming is real, or that it is a conspiracy), so that a pull towards the accurate side could be told apart from agents simply agreeing with whatever is stated. And in control runs the agents did not interact at all and only reported their opinion, which shows how far the model leans one way on a topic even without social influence.

Wang et al. [23] propose a Social Simulation Framework and are the source of the three metrics used throughout this thesis (Section 2.3, their §4.2), which they assemble from earlier work [7, 10]. Every agent in a run is driven by the same model (GPT-4o-mini); their 50 agents are identified only by an index and given a gender, an age and Big Five personality traits. Agents sit on networks of the same three kinds covered in Section 2.1. Each round, an agent reads the opinions of some of its neighbours, by default only those whose opinion is already close to its own (a recommendation rule meant to mimic social-media feeds), then reasons about them and updates its own opinion. They compare these runs with the bounded confidence and Friedkin–Johnsen models on the same networks. Their central finding is that echo chambers (high polarization together with high neighbour correlation) emerge on small-world and scale-free networks but not on random ones, for the LLM agents as for the numeric models; an ablation in their appendix reports that echo chambers still form without the recommendation rule. They also test two "nudges" meant to reduce echo chambers.

Cerina and Duch [5] use LLMs in public-opinion research, but not as simulated people. They collect the recent posts of real users of X (formerly Twitter) before the 2020 US presidential election; a language model (GPT-3.5) reads each user's posts and labels that user with demographics and vote intention, and crowd workers label a sample of the same users to check the model's labels. The labelled sample is then reweighted to the population with multilevel regression and post-stratification, plus a correction for the kind of people who post online, giving state-level vote-share estimates about as accurate as established polling aggregators. The model only measures opinions that real people expressed; it never stands in for them. In their discussion the authors are explicit about the limits of the alternative: views produced by LLMs speaking as synthetic personas are prone to bias, and "there is no reason to expect LLMs' reactions to unfolding events to align with those of real individuals". It is included here for that caution: the opinions of LLM agents, in this thesis as in the studies above, are evidence about the language models, not about real populations.

Piao et al. [22] scale this kind of simulation up substantially: 1000 LLM agents per run interact through model-guided conversations and update their opinions on political issues (partisan alignment, gun control, abortion). Their main results use GPT-3.5; runs driven by other backbone models (GPT-4o, ChatGLM, Llama-3 and DeepSeek-V3) give comparable results. The agents carry no persona, and they report their opinion themselves on a five-level left–right scale, with no separate classifier. Unlike the studies above, the network is not fixed: it starts as an almost regular small-world graph, and then each agent decides whether to talk to someone it is already connected to or to a new, random agent, so the network that matters emerges over the simulation. They report that these self-built networks have properties characteristic of human social networks, including **homophilic clustering** (agents preferentially connecting to others who already agree with them), that polarization emerges, and that the collective dynamics are qualitatively consistent with patterns from empirical studies and opinion-dynamics models, without fitting any numeric rule to the runs. They then test interventions: more diverse interactions, and removing selective exposure or confirmation bias from the agents' prompts.

2.5 Relation to this thesis

Wang et al. [23] is the main reference point for this thesis. disensa reuses their three metrics (Section 2.3) and their general idea of agents exchanging messages on a network, but it is a

separate implementation, not a modification of their code, and its experiments are designed independently of theirs. Wang et al. also compare their LLM agents with the bounded confidence and Friedkin–Johnsen models on the same three kinds of network, by looking at how the metrics evolve under each. disensa takes that comparison further: each LLM run is paired with numeric runs on the exact same network and starting opinions, and the numeric rules are fitted to the LLM run (Chapter 3).

Three design choices set disensa's agents apart from the related work above, and I made all three deliberately, as detailed in Chapter 3. First, **vocabulary**: Chuang et al. and Wang et al. frame their agents as social-media users (Chuang et al.'s agents write tweets; Wang et al. choose what each agent reads with a recommendation rule modelled on social-media feeds), and Piao et al. describe agents forming a social network through conversations; I frame disensa's agents as members of a community who share messages in interactions, with no reference to a specific platform or medium, since nothing about the mechanism studied here depends on resembling any one social-media product. Second, **personas**: Chuang et al. give every agent a demographic persona (name, political leaning, age, gender, ethnicity, education, occupation), and Wang et al. give every agent a gender, an age and Big Five personality traits. Such traits are a second source of differences between agents, on top of their opinions. disensa's agents, like Piao et al.'s, carry no persona: only a minimal identity ("Agent i, a member of a community"), so that the only differences between the agents of a run are their starting opinions and their places in the network, and therefore what they hear. Nothing in the prompt makes one agent more stubborn or more persuadable than another (Chapter 3). Third, **model heterogeneity**: in all three studies every agent in a run is driven by the same model. Chuang et al. repeat some experiments with other models as a robustness check, and Piao et al. run their simulation on five backbone models, but always as separate runs, one model at a time. disensa can place several models side by side as agents of the *same* run, on the same network and the same starting opinions, so that if models differ in how persuadable or how extreme they are, that difference shows up directly in one run rather than only across runs.

Piao et al. [22] take a different direction: their agents build their own network as the simulation runs. disensa keeps the network fixed instead, because a fixed network lets every LLM run be paired, seed for seed, with numeric-rule runs on the *exact same* network and starting opinions (Section 3.5), which is what makes RQ A (Section 1.2) possible to ask in the first place: how far a numeric rule describes an LLM run is only a meaningful comparison when both run on the same world. Model heterogeneity remains a feature of disensa, but it is not used in the experiments of this thesis. As in the studies above, every run is driven by a single model; four open-weight models are compared across separate runs (Section 3.10), so that the answers to RQ A and RQ B are not statements about one model only. Placing different models in the same run would add a further question on top of these, how agents driven by different models influence each other, and is left for future work. The central contribution of this thesis is the framework that pairs LLM and numeric runs directly (Chapter 3), demonstrated on two questions: how far the classic rules describe LLM-agent runs, and how a single LLM agent updates its opinion (RQ A and RQ B, Section 1.2).

Chapter 3
Method: The disensa Framework

This chapter describes disensa, the simulation framework built for this thesis, as the code does it. Sections 3.1–3.6 cover how a run works: the simulation loop shared by both kinds of agents (Section 3.1), the networks and starting opinions (Section 3.2), the numeric agents (Section 3.3), the LLM agents (Section 3.4), how randomness is seeded so that runs can be paired and repeated (Section 3.5), and what happens when a model replies with something unusable (Section 3.6). Sections 3.7–3.8 cover how opinions are measured and how an LLM run is compared with the numeric rules. Sections 3.9–3.11 give the metrics, the design of the experiments and the computational setup. The text describes disensa version 0.3 [TODO: the git tag of the code the experiments ran on].

3.1 Simulation loop

A **run** is one simulation: N agents on a network, each holding an opinion on the same statement, updated over a fixed number of steps T. Everything about a run is set in one configuration file: the network and its parameters, how the starting opinions are drawn, the number of steps, the kind of agents and their settings, and two seeds, `seed` and `replicate` (Section 3.5). The file is checked against a schema before anything runs, so a missing or out-of-range parameter stops the run before it starts. Appendix B lists the configuration files of the experiments.

Every run, numeric or LLM, goes through the same loop (Algorithm 1). A run of T steps records $T + 1$ states: the starting state (step 0) and one after each step.

Algorithm 1 The simulation loop, the same for numeric and LLM agents. Only the lines in blue depend on the kind of agent.

1: Build the network from the `seed` (Section 3.2)
2: Give every agent a starting opinion, drawn from the `seed`
3: LLM agents only: every agent gets a model (Section 3.4.5) and writes a first message
4: Save the starting state as step 0
5: **for** every step $t = 1, \ldots, T$ **do**
6: Every agent picks its new opinion from the state of step $t - 1$: numeric agents apply their rule to their neighbours' opinions; LLM agents hear the messages of a few neighbours and ask their model
7: Any opinion outside $[-2, 2]$ is moved to the nearest end of the scale
8: Compute the metrics, then save the new opinions, the metrics and the step's log
9: **if** too many of this step's model replies failed **then**
10: Stop the run and mark it as *failed* (Section 3.6)
11: **end if**
12: **end for**

Everything except the two blue lines is the same code for both kinds of agents. This is what makes RQ A possible (Section 1.2): an LLM run and a numeric run with the same `seed` start from the same world and differ only in how agents update their opinions. The first messages of line 3 are what the LLM agents hear at step 1. The step's log (line 8) holds one record per agent; for LLM agents it records what the agent heard, its prompt, its reply and any error (Section 3.6).

Updates are **synchronous** for both kinds of agents, as in Section 2.2: every agent works from the same snapshot, the state at step $t-1$, and all new opinions replace the old ones at once. No agent sees another's step-t opinion before the step is over, so the order in which agents are processed never matters. For LLM agents this holds for everything an agent carries, not only its opinion: its message and its memory (Section 3.4) are also replaced for all agents together, after every model call of the step has returned.

Each run gets its own folder, written step by step while it runs, and a status: running, finished, cancelled, or failed (with the reason). A run that stops early, because it was cancelled or because it failed, keeps every step it finished.

3.2 Networks and initial opinions

3.2.1 Networks

disensa builds the three network topologies of Section 2.1 with NetworkX [18], a Python library for building and analysing networks. The agents are the nodes, numbered 0 to $N-1$, and the network stays fixed for the whole run. Every topology is built from the number of agents N and its own parameters, defined in Section 2.1:

- **Erdős–Rényi**: p, the probability that any two agents are linked;
- **Watts–Strogatz**: k, the number of nearest nodes each agent is linked to on the starting ring ($k/2$ on each side, so k is even), and β, the probability that a link is rewired;
- **Barabási–Albert**: m, the number of links each new agent brings when it joins.

Every network in disensa is **connected**. Opinions only travel along links, so two parts of a network with no path between them would be two separate simulations that never influence each other, and some results of Chapter 2, such as DeGroot reaching consensus, would not apply. Of the three topologies, only Erdős–Rényi can come out disconnected: Barabási–Albert is connected by construction, and disensa uses NetworkX's connected variant of Watts–Strogatz, which repeats the rewiring until the result is connected. If an Erdős–Rényi draw is disconnected, disensa draws it again with a new seed derived from the run's `seed` and the attempt number (Section 3.5), up to 100 times. With the settings used in this thesis ($N = 50$, $p = 0.12$), about 9% of first draws are disconnected (93 of 1,000 seeds), so a redraw is common but rarely needed twice. Allowing disconnected networks would also add a difference between topologies that has nothing to do with their structure, since only one of them could split.

3.2.2 The networks of the experiments

Every LLM run of this thesis uses one of three network settings, the same throughout. To compare topologies, the three have the same number of agents and about the same average degree, so that a difference between them comes from their structure rather than from one network being denser than another (Table 2.1). The number of agents is $N = 50$, as in Wang et al. [23]. I set the average degree to $\langle k \rangle \approx 6$, a sparse network in which each agent is linked to about 6 of the 49 others, and derived each topology's parameters from it:

- **Watts–Strogatz**: $k = 6$, since every agent keeps exactly k links on average; $\beta = 0.1$ puts the network in the small-world regime (Figure 2.1);

- **Erdős–Rényi**: $p = 0.12$, from $\langle k \rangle = p\,(N-1)$ (Equation 2.3): $6/49 \approx 0.12$;
- **Barabási–Albert**: $m = 3$, from $\langle k \rangle \approx 2m$.

For Barabási–Albert the average degree comes out slightly below 6, at 5.64. NetworkX starts the network from a small star, one agent linked to m others, and then adds the remaining $N-m-1$ agents with m links each. The star has only m links for its $m+1$ agents, fewer than the m per agent that every later agent brings, so the network has $L = m+m(N-m-1) = 141$ links and $\langle k \rangle = 2L/N = 5.64$. This only affects the total: the agents of the starting star are still the oldest, and so, by preferential attachment, the most likely to become hubs (Section 2.1.4).

Each of the five worlds of the experiments (seeds 1 to 5, Section 3.5) has its own network of each topology, so the experiments use 15 networks in all. Table 3.1 describes them. The table needs no simulation: it is computed from the networks alone, which are fixed by the seeds, before any run.

Table 3.1: The 15 networks of the experiments: each topology built with the five experiment seeds (1–5), $N = 50$. Each column gives the range over the five seeds: average degree $\langle k \rangle$, smallest and largest degree, average clustering $\langle C \rangle$ and average path length $\langle d \rangle$ (Section 2.1.1). The last column counts the seeds whose first draw was disconnected and was redrawn. Computed from the networks alone, before any run.

Network	$\langle k \rangle$	Degrees	$\langle C \rangle$	$\langle d \rangle$	Redrawn
Random, $p = 0.12$	5.56–6.80	1–13	0.11–0.17	2.26–2.46	1
Small-world, $k = 6$, $\beta = 0.1$	6.00	4–8	0.43–0.49	2.80–2.90	0
Scale-free, $m = 3$	5.64	1–26	0.19–0.28	2.24–2.31	0

The table shows the three structures of Chapter 2 at this size. The small-world networks have more than twice the clustering of the other two and nearly equal degrees. The scale-free networks have hubs with up to 26 links. Only the random networks differ in their number of links from seed to seed. In Erdős–Rényi each of the $N(N-1)/2 = 1{,}225$ possible links is present or not by chance, so the number of links is itself random: 147 on average for $p = 0.12$, but 139 to 170 over these five seeds. In the other two topologies the number of links is fixed by construction: rewiring moves links without adding or removing any (150 links for Watts–Strogatz), and every agent of Barabási–Albert brings exactly m (141). Some agents have a single neighbour; Section 3.4 returns to what this means for an LLM agent.

3.2.3 Initial opinions

Each agent starts with an opinion $x_i(0) \in [-2, 2]$, drawn independently for every agent. disensa offers three distributions: **uniform** on $[-2, 2]$; **normal** with a chosen mean and standard deviation; and **bimodal**, where each agent joins one of two camps at $\pm m$ with equal probability and is then spread around it. Draws from the normal and bimodal distributions that fall outside $[-2, 2]$ are clipped to the nearest end. The experiments of this thesis all use the uniform distribution. It is the standard starting point of the numeric models this thesis compares against: Deffuant et al. [11] and Hegselmann and Krause [19] draw the initial opinions uniformly at random, and Lorenz [20] notes that nearly every study of bounded confidence does the same. The closest LLM-agent studies start from it too. Wang et al. [23] use uniform initial beliefs for both their LLM agents and their bounded-confidence baseline "to give a fair comparison", with the same initial agents on every network structure, as disensa does with its seeds. Chuang et al. [8] use it in most experiments as "an unbiased starting state". A uniform start gives no side of the statement an advantage, so a later drift towards one side comes from the dynamics, not from the start. It is also what the two framings of a topic need (Section 3.10): only from a start balanced around 0 can a drift that follows the content be told apart from one that follows the wording. Other starts can matter. Lorenz [20] calls non-uniform starts a very important and still open issue for

bounded confidence. For LLM agents, Chuang et al. [8] and Cau et al. [4] found the agents' main tendency largely unchanged from skewed or polarized starts, but Cau et al. found the outcome from an unbalanced start depended on the model. Other starts are left for future work.

With 50 agents the mean of the starting opinions is close to 0 but not exactly 0; this is why the drift of a run is always measured from its own starting mean (Section 3.8).

Opinions are drawn in the order of the agents, 0 to $N-1$, and so are unrelated to where an agent sits in the network: a hub is no more likely to hold an extreme opinion than any other agent, and neighbours are no more alike than strangers at step 0. Any correlation between neighbours that appears later is produced by the dynamics. The draw has its own random stream, fixed by `seed` (Section 3.5), so a numeric run and an LLM run with the same `seed` start from exactly the same opinions on exactly the same network.

3.3 Numeric agents

Numeric agents apply one of the three rules of Section 2.2: DeGroot (Equation 2.8), bounded confidence in its inertial Hegselmann–Krause form (Equation 2.10), or Friedkin–Johnsen (Equation 2.11). The equations in the code are the ones given there. Each rule is written for the whole population at once, as operations on the adjacency matrix A and the opinion vector $x(t-1)$, so a step is a few matrix operations with no randomness. Table 3.2 lists the parameters.

Table 3.2: Parameters of the numeric rules. The defaults apply only when a configuration file does not set a value; the experiments set every value explicitly.

Rule	Parameter	Range	Default
DeGroot	none	—	—
Bounded confidence	confidence bound ε	$\varepsilon > 0$	1.0
	step size μ	$0 < \mu \leq 1$	0.5
Friedkin–Johnsen	stubbornness λ	$\lambda > 0$	1.0

Three details of the implementation are worth stating. First, in bounded confidence an agent always counts itself in its confidence set (Equation 2.9), since its distance to its own opinion is 0; the average is therefore always defined, and an agent with no neighbour within ε simply stays where it is. Second, the innate opinion s_i of Friedkin–Johnsen is the agent's starting opinion $x_i(0)$, stored at the start of the run and fixed for the rest of the run. Third, every rule averages opinions that already lie in $[-2, 2]$, so the new opinion is always inside the scale too, and the clipping step of Algorithm 1 never changes a numeric run.

Numeric agents read the exact opinion of every neighbour, at every step. This is the main difference from LLM agents, which hear only some of their neighbours and only their words (Section 3.4). A numeric run is fully determined by its world (its network and starting opinions). The second seed of a run, the replicate, sets only the random draws made during the run (Section 3.5). The numeric rules make no such draws, so the replicate has no effect on a numeric run, and disensa warns if it is set.

Numeric agents are used in two ways in this thesis. They run on their own, as reference runs on the same worlds as the LLM runs (Chapter 4). And they are fitted to each LLM run: the same rule code is run again on the LLM run's network and starting opinions, over a grid of parameter values (Section 3.8).

3.4 LLM agents

An LLM agent replaces the update equation with a language model. Instead of computing its new opinion from numbers, it reads what a few of its neighbours said, reasons about it,

and replies with a new opinion, a new message and an updated memory. The mechanism is the one of Wang et al. [23], framed differently and without personas (Section 2.5). Figure 3.1 shows one step of one agent; the rest of this section goes through it. Every prompt is quoted in full in Appendix A.

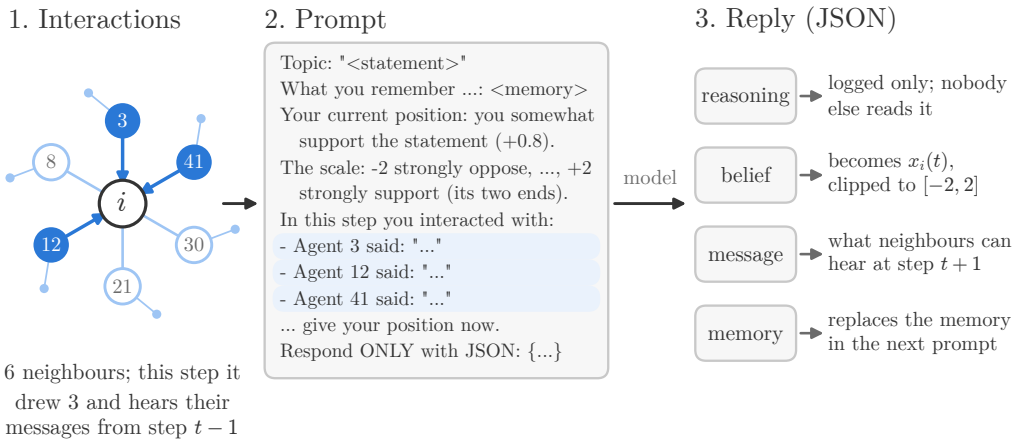


Figure 3.1: One step of one LLM agent. (1) Agent i draws how many of its neighbours it hears this step, here 3 of 6, and which ones; it hears the message each of them shared at step $t-1$. (2) Those messages go into its prompt, together with the statement, its memory and its current position (abridged here; the full prompt is in Appendix A). (3) The model replies with four fields, and each is used in a different way. Every agent does this from the same state at step $t-1$, and all new states are applied together.

3.4.1 What an agent is

An LLM agent is a node of the network with a model attached. Three things carry over from one step to the next:

- its **opinion** $x_i(t) \in [-2, 2]$, exactly as for numeric agents, and the only part of the state the metrics look at;
- its **message**: the last thing it said, 1–3 sentences, which is what its neighbours can hear at the next step;
- its **memory**: a short summary of what it has heard and thought so far, which the agent rewrites itself at every step.

A call to a language model is **stateless**: the model remembers nothing from one call to the next, so everything an agent "knows" at step t has to be in the prompt of that call. Its identity is minimal. The system prompt, the part of the prompt that sets the model's role, is the same for every agent and every step except for the agent's number:

```
You are Agent {agent_id}, a member of a community. You interact with
other people and share your views with them. Think and speak as this
person would, in plain everyday language.
```

There is no name, age, gender or personality. As explained in Section 2.5, the agents of a run differ only in their starting opinions and their places in the network, and nothing in the prompt makes one agent more stubborn or more persuadable than another. The prompts describe a community exchanging messages, with no reference to any platform or medium.

3.4.2 Step 0: the first message

An LLM agent does not choose its starting opinion: it gets $x_i(0)$, drawn as for numeric agents (Section 3.2). This keeps LLM and numeric runs comparable, since both start from the same opinions. The cost is that a model's own leaning on the statement is hidden at the start, and can only show up in later steps.

The opinion is stored as an exact number, which is never rounded. The prompt shows it to the model in two forms, each rounded only for display. The first is one of five **stance labels** (Table 3.3), chosen by rounding the opinion to the nearest whole number.[1] The second is the number itself, rounded to one decimal. An opinion of -0.8347, for example, is shown as "somewhat oppose" (from -1) and as -0.8. At step 0 every agent is asked for a first message, for example: "You somewhat oppose this statement (your position: -0.8)", followed by the five points of the scale and the instruction to write a short message of 1–3 sentences stating its view, as JSON. These first messages are what the agents hear at step 1. Step 0 changes no opinion.

Table 3.3: The stance labels: how an opinion is put into words in the prompts. The same five points name the scale in every prompt.

Opinion (rounded)	Label
-2	strongly oppose
-1	somewhat oppose
0	neutral / undecided
$+1$	somewhat support
$+2$	strongly support

3.4.3 Interactions

At each step $t \geq 1$, an agent **interacts** with a few of its neighbours: it hears the message each of them shared at step $t - 1$. Who it hears is drawn in two parts, separately for every agent and anew at every step:

1. **how many**: a whole number drawn uniformly between a minimum and a maximum (1 and 5 in the experiments), and capped at the number of neighbours that have a message. An agent with 3 neighbours that draws 5 hears all 3;

2. **which ones**: that many neighbours, drawn uniformly at random without repetition. Every neighbour is equally likely, whatever its opinion or its degree.

Hearing is therefore not mutual: agent i can hear j at a step without j hearing i. An agent hears at most five others per step, however many neighbours it has, but a hub can be heard by many. An agent with a single neighbour, which exists in the random and scale-free worlds (Table 3.1), hears that neighbour at every step. Each message reaches the prompt as one line, `- Agent 7 said: "..."`, so an agent knows who said what but never anyone's number: only the words of a neighbour reach it.

Choosing neighbours uniformly is a deliberate difference from Wang et al. [23], whose agents by default read only neighbours whose opinion is already close to their own (Section 2.4). That rule builds a form of bounded confidence into what an agent hears. In disensa, nothing in the choice of interactions favours similar opinions, so if LLM agents end up listening mainly to agents that already agree with them, the model did that, not the code.

With a maximum of 0, nobody is heard: the prompt is the same, with "(nobody in this step)" in place of the messages. This is the **no-interaction control** of Chuang et al. [8]:

[1] A value exactly halfway between two whole numbers is rounded to the even one, Python's default: ± 0.5 becomes 0 (neutral) and ± 1.5 becomes ± 2 (strongly). Starting opinions are practically never exactly halfway, but a belief reported by a model can be.

the agents are only asked for their opinion again at every step, so whatever they still move is the model's own drift, not the influence of other agents (Section 3.10).

3.4.4 Updating the opinion

The prompt of step t gives the agent the statement, its memory ("(nothing yet)" at step 1), its current position as a label and a value, the five points of the scale, and the messages it heard. It asks the agent to "reflect on these interactions, as the person you are, and give your position now". The prompt says that -2 and $+2$ are the ends of the scale, and that the belief is a number from -2 to 2 with decimals allowed. It does not suggest which way to move, or whether to move at all. The reply must be a JSON object with four fields, each used differently (Table 3.4).

Table 3.4: The four fields of an LLM agent's reply at steps $t \geq 1$.

Field	What the prompt asks for	What happens to it
`reasoning`	2–4 sentences on how the interactions affect its view	Logged only; never shown to any agent
`belief`	Its new position, a number from -2 to 2	Becomes $x_i(t)$, clipped to $[-2, 2]$
`message`	Its new message, 1–3 sentences	What neighbours can hear at step $t+1$
`memory`	An updated summary, at most 60 words	Replaces the memory in the next prompt

The lengths in Table 3.4 are requests in the prompt, not limits enforced by the code: a longer message or memory is kept as it is. The only hard limit is the length of the whole reply, 400 tokens. A reply cut off at that limit is usually incomplete JSON, and is handled as described in Section 3.6.

There is **no update rule** in this process. Unlike the numeric rules, there is no parameter that shapes the update: no confidence bound, no step size, no stubbornness. The new opinion is whatever number the model reports. If LLM agents end up ignoring distant opinions, as under bounded confidence, or holding on to where they started, as under Friedkin–Johnsen, that behaviour comes from the model. Asking how far it matches those rules is RQ A; asking what drives it, agent by agent, is RQ B. The opinion is also **self-reported**: it is the number the agent says it holds. Whether that number matches what its message expresses is a separate question, taken up in Section 3.7.

3.4.5 Model assignment

The models of a run are given as a list, each with a **weight**: its share of the agents. The shares are turned into whole numbers of agents that always add up to N, using **largest-remainder rounding**: each model first gets the whole part of its exact share, and the agents left over go one each to the models with the largest fractional parts. For example, three models with equal weight and 20 agents get 7, 7 and 6. The resulting list of models is then shuffled and handed out to the agents, so the models are placed on the network at random. The shuffle has its own random stream, fixed by `seed` alone (Section 3.5), so two runs with the same `seed` and different models still have the same network, the same starting opinions and the same interactions; only the models differ. As explained in Section 2.5, every run in this thesis uses a single model, with weight 1; mixing models in one run is left for future work.

disensa sends every request through LiteLLM,[2] a library that talks to many model servers in the same way. A model is named by its server and its name there: `ollama_chat/<model>`

[2] `https://github.com/BerriAI/litellm`

for a model served by Ollama[3] on the same computer, and `hosted_vllm/<model>` with the server's address for a model served by vLLM on a GPU server (Section 3.11). In both cases the models are open-weight models run on hardware I control, so no data goes to a model provider. At the start of each run disensa also asks every server what exactly it is serving (for Ollama, the model's digest and quantization; for vLLM, the model identifiers) and records the answer with the run, because the same model name can stand for different weights on different servers.

Every request also carries settings that are not part of the prompt. A language model writes its reply one token (a word or part of a word) at a time, drawing each from a probability distribution over possible next tokens. The **temperature** reshapes that distribution: lower values make the most likely token win more often and the replies more alike, higher values let less likely tokens through. **top-k** and **top-p** narrow the draw further: only the k most likely tokens, and among them only the smallest set whose probabilities add up to p, can be picked. disensa sends the temperature (0.7), top-p (0.9), top-k (40), the maximum reply length and a sampling seed (Section 3.5) with every request, instead of leaving any of them to the server. Ollama and vLLM have different defaults, so a setting left to the server would make runs on the two differ for a reason nobody chose. Every model gets the same prompts and the same settings. The same temperature does not make two models equally random, though: it reshapes each model's own probabilities in the same way, it does not make the models behave alike.

Some models can "think" before answering: they write hidden reasoning first, then the reply. That reasoning uses up the reply length, can confuse the reading of the JSON, and is slow, so disensa switches it off on every request (on Ollama with its own `think` option, on vLLM with the chat-template switch of the models that have one). If a reply still starts with a block of hidden reasoning, the block is removed before the JSON is read. The switch is recorded with the run, like every other setting.

3.5 Seeding and reproducibility

Every random draw in a run is fixed by two numbers (Table 3.5). The **seed** fixes the **world**: the network, the starting opinions and which agent gets which model. The **replicate** (default 0) fixes the **dynamics**: whom each agent hears at each step, and the sampling seed sent with every model call.

Table 3.5: What fixes each random draw of a run.

Draw	Fixed by	Part of
The network	`seed`	the world
The starting opinions $x(0)$	`seed`	the world
Which agent gets which model	`seed`	the world
Whom each agent hears, at every step	`seed` and `replicate`	the dynamics
The sampling seed of the model calls	`seed` and `replicate`	the dynamics

This split gives the two comparisons the experiments need. With the **same seed and different dynamics**, an LLM run and a numeric run start from exactly the same network and opinions, so the numeric rules can be compared with that very LLM run, state for state (Section 3.8). It also pairs runs across conditions: every statement, topology and model meets the same worlds, so a difference between two conditions is never a difference between their worlds. With the **same seed and a different replicate**, two LLM runs share their world and differ only in whom the agents hear and in the model's own sampling. How far their outcomes differ measures how much of an outcome is chance rather than the world; this is the replicate reference of RQ A (Section 1.2).

[3] `https://ollama.com`

Each draw comes from its own random stream. disensa derives the streams with NumPy's `SeedSequence`, which turns a list of numbers into a stream statistically independent of the streams made from other lists; each stream's list holds the `seed` (and, for the dynamics, the `replicate`) together with a number identifying the stream. No two draws share random numbers, so changing one of them, for example the range of the number of interactions, does not change the others. The network is the one exception to the list above: NetworkX takes the `seed` itself, and the redraws of a disconnected network (Section 3.2) use seeds derived from the `seed` and the attempt number, rather than simply the next seed, so that the redraw of one world does not reuse the network of another.

The **sampling seed** is one number per run, derived from the `seed` and the `replicate`, and sent with every model call of the run. With the same seed, a server that is otherwise deterministic gives the same reply to the same request; replies within a run still differ, because no two prompts of a step are the same (each holds the agent's number, its opinion, its memory and what it heard). A different replicate changes the sampling seed and so gives the model a fresh draw.

Every reply is also **cached** on disk under its exact request: the model, the full prompt, and every setting sent with it (temperature, top-p, top-k, the thinking switch, the maximum reply length and the sampling seed). Running the same configuration again replays the cached replies instead of calling the model, so an LLM run that has been run once can be reproduced exactly, and an interrupted run that is started again does not pay a second time for the replies it already had. Changing anything in a request, such as the statement, a word of the prompt or a neighbour's message, makes it a new request with no cached reply. The address of the model server is not part of the request, so the same model served on another machine reuses the same cache. Without the cache, a rerun on a GPU server is not guaranteed to be identical even with the same sampling seed, because the server may group requests differently (Section 3.11); the cache and the per-step log keep exactly what was generated.

Each run folder records what is needed to repeat it: the full configuration, the git commit of the code, the version of the seeding scheme, the version of disensa, and, for LLM runs, what each model server reported serving.

3.6 Malformed output and the run report

A language model does not always reply in the requested format, and a model server can fail. disensa treats every reply as untrusted: a bad reply must never crash a run, and it must never pass silently either.

3.6.1 Reading a reply

A reply goes through four stages. First, disensa asks the server for **JSON mode**, which constrains the reply to valid JSON; disensa requests it from both Ollama and vLLM. Second, the reply is **parsed**: a leading block of hidden reasoning is removed, and the first JSON object in the text is extracted, which tolerates text around it and a common slip of small models, a comma after the last item. Third, if no JSON object can be read, the model is asked **once more**, with its own reply and the instruction "That was not valid JSON. Reply again with ONLY the JSON object." Fourth, the `belief` is checked: it must be present, and it must be a finite number. A request that fails at the server, for example because it is unreachable or busy, is retried up to four times, waiting 1, 2, 4 and then 8 seconds. The one server error that is not retried is Ollama stopping a model that keeps repeating itself: with the sampling seed fixed, the model would repeat itself again on every retry.

A reply that is still unusable is a **failed reply**, of one of four kinds (Table 3.6). An agent with a failed reply **keeps its previous state** for that step: its opinion, its message and its memory. The run goes on, and the failure is logged with the step, with its kind and the raw reply, so it can be counted and read later. A reply that is valid but lacks a message or a

memory keeps the previous one for that field only. Step 0 is the one case with nothing to keep: an agent whose first message fails has no message yet, so its neighbours cannot hear it until it has said something.

Table 3.6: The kinds of failed reply. In every case the agent keeps its previous opinion, message and memory for that step.

Kind	Meaning
`provider`	The model server returned an error or could not be reached, after the retries
`invalid_json`	No JSON object in the reply, even after asking the model to fix it
`missing_belief`	Valid JSON without a `belief`
`invalid_belief`	A `belief` that is not a finite number (for example `NaN`, `true`, or `"a lot"`)

A belief outside $[-2, 2]$ is *not* a failed reply. It is clipped to the nearest end of the scale (a reply of 2.5 becomes $+2$), and the event records both that it was clipped and the number the model gave. Such beliefs are counted in the run report as **out of range**. The count is reported rather than hidden, because a model that keeps placing itself beyond the ends of a scale it was told about is itself a finding about how well it follows the scale.

3.6.2 Why failures need a limit

A failed reply keeps the agent's opinion. A few failures are normal, but many are easy to miss, because a run where most replies fail looks exactly like a run of very stubborn agents. disensa therefore stops a run when more than a share `max_error_rate` of the replies of one step fail, step 0 included (the check in Algorithm 1). The run is marked *failed*, with the reason, for example "step 3: 47/50 replies failed (provider: connection refused)", and every finished step is kept. The default share is 20%. The experiments use 50%, so that a run stops only when the server is clearly broken; every run below that limit is kept, reports its own share of failed replies, and the analysis checks how much those failures matter (Section 3.10).

3.6.3 The run report

Every run reports how far it can be trusted and what it cost, at three levels: per model call (in the event log), per step (with the metrics) and per run (the **run report**). The report gives:

- **failures**: how many replies failed, by kind and by model;
- **cost**: tokens sent and received, wall-clock time, seconds per model call, and how many replies came from the cache;
- **out of range**: how many beliefs were clipped;
- **rounding**: how many of the valid beliefs were whole numbers (1, -2), halves (0.5, -1.5) or anything finer (0.7, 1.25). A model that only ever replies with whole numbers is using a five-point scale, however finely the prompt allows it to answer, and this limits how small a move can be seen (Section 3.7).

Table 3.7 lists what a run folder holds. The event log is what makes RQ B possible: for every agent at every step it keeps whom the agent heard and what they said, its reasoning, its new message and memory, the exact prompt and raw reply, and any failure. A single agent's change of opinion can therefore be traced back to what it read and what it said about it.

Table 3.7: The files of a run folder. Tables are stored as Parquet files, text as JSON lines (one JSON object per line). Every step is written as soon as it is finished.

File	Contents
`meta.json`	Configuration, git commit, seeding scheme, status (and reason if failed), what each model server served, and the run report
`agents.parquet`	One row per agent: model or rule, degree, starting opinion
`edges.json`	The network, as a list of links
`opinions.parquet`	One row per agent and step: the opinion
`metrics.parquet`	One row per step: the metrics (Section 3.9), failed replies, out-of-range beliefs, tokens and time
`events.jsonl`	LLM runs only. One line per agent and step: whom it heard and their messages, its reasoning, message and memory, the prompt and raw reply, any failure and its kind, the belief before clipping, and the cost of the call
`scores.parquet`	Added after the run by the scorer (Section 3.7); never read by the run

3.7 Measuring opinions: self-rating, scorer, human validation

[Not yet drafted.]

3.8 Comparing LLM runs with numeric rules

[Not yet drafted.]

3.9 Metrics

[Not yet drafted.]

3.10 Experimental design

[Not yet drafted.]

3.11 Computational setup

[Not yet drafted.]

Chapter 4
Experiments and Results

[TODO: one short paragraph once experiments are run, previewing what this chapter shows and how it is organised.]

4.1 Design overview

[TODO] Recaps the experimental design from Section 3.10: the topics, networks, seeds and models actually used, once those are no longer placeholders (`configs/sweeps/topics_core.yaml`).

4.2 RQ A: how far do the classic rules describe LLM runs?

[TODO] Will show, per topic type, how close each of the three classic rules (DeGroot, bounded confidence, Friedkin–Johnsen), fitted to each LLM run on its own world, gets to the run on the distances of Section 3.8 (Wasserstein distance, cluster counts, drift, polarization curve), relative to the nobody-moves baseline and to the run-to-run variation between replicates; which runs no rule describes; and whether any of this changes with topic type (political, moral, factual, trivial).

4.3 RQ B: how do individual agents update their opinions?

[TODO] Will trace individual agents through their logged interactions and reasoning (`events.jsonl`), using the response-curve analysis of Section 3.8 to show how far an agent moves against how far the opinions it heard were from its own, with each classic rule's prediction drawn only as a reference; the control runs without interactions, to separate the model's own drift from what the interactions add; and qualitative examples of agents that moved or held firm, with their reasoning (Appendix A for the prompts, Appendix D for example messages, once added).

4.4 Numeric baseline sweeps

[TODO] Will report the two standalone numeric sweeps that calibrate the rules themselves, independent of any LLM run: final opinion clusters against the bounded-confidence bound ε (`configs/sweeps/numeric_bc_epsilon.yaml`, 200 runs) and final polarization against the Friedkin–Johnsen stubbornness λ (`configs/sweeps/numeric_fj_stubbornness.yaml`, 80 runs).

Chapter 5
Discussion and Limitations

[**TODO**] Short paragraph once Chapter 4 is complete, previewing what this chapter covers.

5.1 Interpretation of the results

[**TODO**] What the RQ A and RQ B results (Chapter 4) actually mean, read together: where a classic rule describes a run, does that say something about the mechanism behind an LLM agent's updates, or only about the outcome it produces; and where none does, what the LLM agents do instead.

5.2 Self-rating versus scorer

[**TODO**] Whether the human validation (Section 3.7) favoured the self-rating or the scorer, and whether that choice changes any of the RQ A/RQ B conclusions.

5.3 Limitations

[**TODO**] Points to cover, at minimum:

- what an LLM-agent simulation can and cannot be taken as evidence of: a result here may say more about the language model's own training and priors than about how real people form opinions, and treating it as the latter would overclaim;
- a degree of circularity worth naming directly: the macro pattern a run produces emerges from micro-level interactions, but those interactions are themselves shaped by whatever the model already encodes about plausible macro outcomes, which makes causal claims about *why* a pattern appears hard to support from the simulation alone;
- the network sizes used here (tens of agents) are small enough that the asymptotic network properties of Section 2.1 hold only approximately;
- the topic statements used for RQ A (political, factual, trivial) were placeholders at the time of writing (`configs/sweeps/topics_core.yaml`) and the specific statements chosen may not be representative of their category;
- disensa deliberately omits personas (Section 3.4), which keeps model identity as the only source of variation between agents, but also means the results may not transfer to settings where agents are given personas;
- Ollama and vLLM serve the same model name at different quantization levels (Section 3.11), so results are only comparable within one serving setup.

5.4 Reproducibility

[TODO] How far the reported runs can be reproduced exactly (Section 3.5, the LLM reply cache) versus how far the qualitative pattern should be expected to generalise to a different seed, model version or provider.

Chapter 6
Conclusion

6.1 Review of Project Goals

[TODO] Whether RQ A and RQ B (Section 1.2) were answered, and to what extent, once Chapter 4 is complete.

6.2 Context

[TODO] How the results fit back against the related work of Chapter 2 (Wang et al., Chuang et al., Piao et al.) and what they add to it.

6.3 Future Works

[TODO] Candidates to cover: the model-heterogeneity direction not pursued in depth here (Section 2.5), topics beyond the four categories used, and any limitation from Section 5.3 that points to a concrete next step.

Appendix A
Prompts

[Not yet drafted. Will quote every prompt template in `src/disensa/llm/prompts.py` verbatim: the system prompt, the first-message prompt, the per-step update prompt, and the scorer prompt.]

Appendix B
Configurations and Seeds

[Not yet drafted. Will list the configuration files and seeds behind every run reported in Chapter 4: `configs/sweeps/topics_core.yaml`, `configs/sweeps/numeric_bc_epsilon.yaml`, `configs/sweeps/numeric_fj_stubbornness.yaml`, once experiments are run.]

References

[1] Albert-László Barabási. *Network Science*. Cambridge University Press, 2016. (Cited on pages 7 and 9)

[2] Albert-László Barabási and Réka Albert. Emergence of scaling in random networks. *Science*, 286:509–512, 1999. (Cited on page 8)

[3] Alain Barrat and Martin Weigt. On the properties of small-world network models. *The European Physical Journal B*, 13:547–560, 2000. (Cited on page 8)

[4] Erica Cau, Valentina Pansanella, Dino Pedreschi, and Giulio Rossetti. Language-driven opinion dynamics in agent-based simulations with LLMs, 2025. arXiv preprint arXiv:2502.19098. (Cited on page 20)

[5] Roberto Cerina and Raymond M. Duch. Artificially intelligent opinion polling. *Royal Society Open Science*, 13:251150, 2026. (Cited on page 15)

[6] Bernard Chazelle and Chu Wang. Inertial Hegselmann–Krause systems. *IEEE Transactions on Automatic Control*, 62(8):3905–3913, 2017. (Cited on page 11)

[7] Uthsav Chitra and Christopher Musco. Analyzing the impact of filter bubbles on social network polarization. In *Proceedings of the 13th ACM International Conference on Web Search and Data Mining (WSDM 2020)*, pages 115–123, 2020. (Cited on pages 12, 13, and 15)

[8] Yun-Shiuan Chuang, Agam Goyal, Nikunj Harlalka, Siddharth Suresh, Robert Hawkins, Sijia Yang, Dhavan Shah, Junjie Hu, and Timothy T. Rogers. Simulating opinion dynamics with networks of LLM-based agents. In *Findings of the Association for Computational Linguistics: NAACL 2024*, pages 3326–3346, 2024. (Cited on pages 3, 14, 19, 20, and 22)

[9] Matteo Cinelli, Gianmarco De Francisci Morales, Alessandro Galeazzi, Walter Quattrociocchi, and Michele Starnini. The echo chamber effect on social media. *Proceedings of the National Academy of Sciences*, 118(9):e2023301118, 2021. (Cited on page 14)

[10] Federico Cinus, Marco Minici, Corrado Monti, and Francesco Bonchi. The effect of people recommenders on echo chambers and polarization. In *Proceedings of the International AAAI Conference on Web and Social Media (ICWSM 2022)*, volume 16, 2022. (Cited on pages 14 and 15)

[11] Guillaume Deffuant, David Neau, Frédéric Amblard, and Gérard Weisbuch. Mixing beliefs among interacting agents. *Advances in Complex Systems*, 3:87–98, 2000. (Cited on pages 11 and 19)

[12] Morris H. DeGroot. Reaching a consensus. *Journal of the American Statistical Association*, 69(345):118–121, 1974. (Cited on page 11)

[13] Paul Erdős and Alfréd Rényi. On random graphs I. *Publicationes Mathematicae Debrecen*, 6:290–297, 1959. (Cited on page 7)

[14] Andreas Flache, Michael Mäs, Thomas Feliciani, Edmund Chattoe-Brown, Guillaume Deffuant, Sylvie Huet, and Jan Lorenz. Models of social influence: Towards the next frontiers. *Journal of Artificial Societies and Social Simulation*, 20(4):2, 2017. (Cited on pages 10 and 11)

[15] John R. P. French. A formal theory of social power. *Psychological Review*, 63(3):181–194, 1956. (Cited on page 12)

[16] Noah E. Friedkin and Eugene C. Johnsen. Social influence and opinions. *Journal of Mathematical Sociology*, 15(3–4):193–206, 1990. (Cited on page 12)

[17] Benjamin Golub and Matthew O. Jackson. Naïve learning in social networks and the wisdom of crowds. *American Economic Journal: Microeconomics*, 2(1):112–149, 2010. (Cited on page 11)

[18] Aric A. Hagberg, Daniel A. Schult, and Pieter J. Swart. Exploring network structure, dynamics, and function using NetworkX. In *Proceedings of the 7th Python in Science Conference (SciPy 2008)*, pages 11–15, 2008. (Cited on pages 10 and 18)

[19] Rainer Hegselmann and Ulrich Krause. Opinion dynamics and bounded confidence: Models, analysis and simulation. *Journal of Artificial Societies and Social Simulation*, 5(3):2, 2002. (Cited on pages 11 and 19)

[20] Jan Lorenz. Continuous opinion dynamics under bounded confidence: A survey. *International Journal of Modern Physics C*, 18(12):1819–1838, 2007. (Cited on page 19)

[21] Cameron Musco, Christopher Musco, and Charalampos E. Tsourakakis. Minimizing polarization and disagreement in social networks. In *Proceedings of the 2018 World Wide Web Conference (WWW 2018)*, pages 369–378, 2018. (Cited on page 13)

[22] Jinghua Piao, Zhihong Lu, Chen Gao, Fengli Xu, Qinghua Hu, Fernando P. Santos, Yong Li, and James Evans. Emergence of polarization in networks of large language model agents. *Nature Communications*, 17:11463, 2026. (Cited on pages 15 and 16)

[23] Chenxi Wang, Zongfang Liu, Dequan Yang, and Xiuying Chen. Decoding echo chambers: LLM-powered simulations revealing polarization in social networks. In *Proceedings of the 31st International Conference on Computational Linguistics (COLING 2025)*, pages 3913–3923, 2025. (Cited on pages 3, 13, 14, 15, 18, 19, 21, and 22)

[24] Duncan J. Watts and Steven H. Strogatz. Collective dynamics of 'small-world' networks. *Nature*, 393:440–442, 1998. (Cited on page 8)

[25] Uri Wilensky and William Rand. *An Introduction to Agent-Based Modeling: Modeling Natural, Social, and Engineered Complex Systems with NetLogo*. MIT Press, 2015. (Cited on pages 3 and 6)